Algoritmi e Strutture Dati – Cap. 3: Liste Collegate

Summary su Record, Puntatori e Operazioni Fondamentali

Riferimento: Demetrescu, Finocchi, Italiano. Algoritmi e Strutture Dati – Seconda Edizione

1 Introduzione

Una **lista collegata** (o concatenata) è una struttura dati dinamica che rappresenta una sequenza ordinata di elementi. A differenza degli array, non richiede un blocco contiguo di memoria: gli elementi possono essere allocati in posizioni qualsiasi della memoria e collegati esplicitamente tramite puntatori.

1.1 Struttura del Record (Nodo)

Ogni elemento della lista è memorizzato in un **record** (o nodo) composto da:

- `chiave` (o `data`): il dato o valore contenuto nel nodo.
- `next`: un puntatore al nodo successivo nella sequenza (oppure `null` se è l'ultimo).
- `prev` (solo per liste doppiamente collegate): un puntatore al nodo precedente.

1.2 Tipologie di Liste

1. **Lista Monodirezionale (Singly Linked List):** Ogni nodo contiene solo il puntatore `next`. La scansione può avvenire solo in avanti.
2. **Lista Bidirezionale (Doubly Linked List):** Ogni nodo contiene i puntatori `next` e `prev`, permettendo l'attraversamento in entrambe le direzioni.
3. **Liste Circolari:** L'ultimo nodo punta al primo nodo (e il primo al precedente ultimo, nel caso bidirezionale).

2 Confronto con gli Array

Operazione / Proprietà	**Array**	**Lista Collegata**
Accesso diretto (i-esimo elemento)	$O(1)$	$O(n)$
Inserimento/Cancellazione in testa	$O(n)$	$O(1)$
Inserimento/Cancellazione in posizione nota	$O(n)$ (shift elementi)	$O(1)$
Ricerca di un elemento	$O(n)$ ($O(\log n)$ se ordinato)	$O(n)$
Uso della memoria	Contiguo, dimensione fissa	Dinamico, extra overhead (puntatori)

3 Operazioni Fondamentali ed Analisi dei Casi

Nelle liste collegate, le prestazioni delle operazioni dipendono in modo fondamentale da due fattori:

a) **La posizione** dell'elemento coinvolto (in testa, in mezzo, in coda).

b) **I parametri forniti in ingresso** (se viene fornita soltanto la chiave k, oppure il riferimento/puntatore diretto ad un nodo o al predecessore).

3.1 1. Ricerca: `search(k)`

Cerca un nodo con chiave k partendo dalla testa della lista.

- **In testa:** $O(1)$ (trovato immediatamente al primo passo per entrambe le tipologie di liste).
- **In mezzo / In coda / Non presente:** $O(n)$ nel caso peggiore e medio per entrambe le tipologie, poiché occorre scorrere sequenzialmente la lista.

Algoritmo `Search`(L, k):

1. $curr \leftarrow L.head$
2. **while** $curr \neq$ `null` **and** $curr.key \neq k$ **do**
3. $\quad curr \leftarrow curr.next$
4. **return** $curr$

3.2 2. Inserimento: `insert`

L'operazione varia a seconda dei parametri passati in ingresso:

3.2.1 Caso A: Inserimento data la sola chiave di riferimento (`insert(elem, key_rif)`)

Se viene fornita solo la chiave del nodo dopo cui inserire il nuovo elemento, occorre prima individuare tale nodo tramite ricerca:

- **In testa (`insertFirst`):** $O(1)$ sia per Singly Linked che per Doubly Linked (nessuna ricerca necessaria).
- **In mezzo o in coda:** $O(n)$ per entrambe le tipologie, dominato dal costo della ricerca della chiave di riferimento.
- **In coda con puntatore `tail`:** $O(1)$ per entrambe le tipologie, se la struttura mantiene un puntatore esplicito all'ultimo elemento. In caso non ci sia un puntatore a `tail`, la complessità è $O(n)$.

3.2.2 Caso B: Inserimento dato il puntatore al predecessore (`insertAfter(elem, pred)`)

Se si dispone direttamente del puntatore al nodo precedente `pred` (o si inserisce in testa con `pred` = `null`):

- **Singly Linked List:** $O(1)$ in qualsiasi posizione (testa, mezzo, coda), poiché basta riassegnare i puntatori locali `next`.
- **Doubly Linked List:** $O(1)$ in qualsiasi posizione, aggiornando localmente sia i puntatori `next` che `prev`.

Algoritmo `InsertFirst`(L, x) – Inserimento in Testa (Doubly Linked):

1. $x.next \leftarrow L.head$
2. $x.prev \leftarrow$ `null`
3. **if** $L.head \neq$ `null` **then** $L.head.prev \leftarrow x$
4. $L.head \leftarrow x$

3.3 3. Cancellazione: `delete`

3.3.1 Caso A: Cancellazione data solo la chiave (`deleteByKey(k)`)

Dovendo rimuovere un elemento conoscendone solo la chiave k:

- **In testa:** $O(1)$ sia per Singly Linked che per Doubly Linked.
- **In mezzo o in coda:** $O(n)$ per entrambe le tipologie, a causa della ricerca preliminare necessaria per trovare il nodo e il suo predecessore.

3.3.2 Caso B: Cancellazione dato il puntatore diretto al nodo x (`delete(x)`)

- **Doubly Linked List:**
  - **In testa:** $O(1)$.
  - **In mezzo o in coda:** $O(1)$. Sfruttando i puntatori $x.prev$ e $x.next$, la rimozione richiede solo il riallineamento locale dei vicini.
- **Singly Linked List:**
  - **In testa:** $O(1)$ (basta aggiornare $L.head \leftarrow x.next$).
  - **In mezzo o in coda:** $O(n)$, poiché senza il puntatore `prev` è necessario scorrere la lista dall'inizio per trovare il predecessore `pred` tale che `pred.next` $= x$.
  - **Se viene fornito anche il predecessore `pred`:** $O(1)$ in qualsiasi posizione.

Algoritmo `Delete`(L, x) – Lista Bidirezionale ($O(1)$):

1. **if** $x.prev \neq$ `null` **then** $x.prev.next \leftarrow x.next$
2. **else** $L.head \leftarrow x.next$
3. **if** $x.next \neq$ `null` **then** $x.next.prev \leftarrow x.prev$

4 Riepilogo Dettagliato delle Complessità

Operazione / Posizione	Parametri presi	Singly Linked		Doubly Linked	
		Testa	Mezzo / Coda	Testa	Mezzo / Coda
Search	Solo chiave k	$O(1)$	$O(n)$	$O(1)$	$O(n)$
Insert	Solo chiave di rif.	$O(1)$	$O(n)$	$O(1)$	$O(n)$
	Puntatore `pred`	$O(1)$	$O(1)$	$O(1)$	$O(1)$
	Con puntatore `tail` (in coda)	–	$O(1)$	–	$O(1)$
Delete	Solo chiave k	$O(1)$	$O(n)$	$O(1)$	$O(n)$
	Puntatore al nodo x	$O(1)$	$O(n)$	$O(1)$	$O(1)$
	Puntatore x e `pred`	$O(1)$	$O(1)$	$O(1)$	$O(1)$